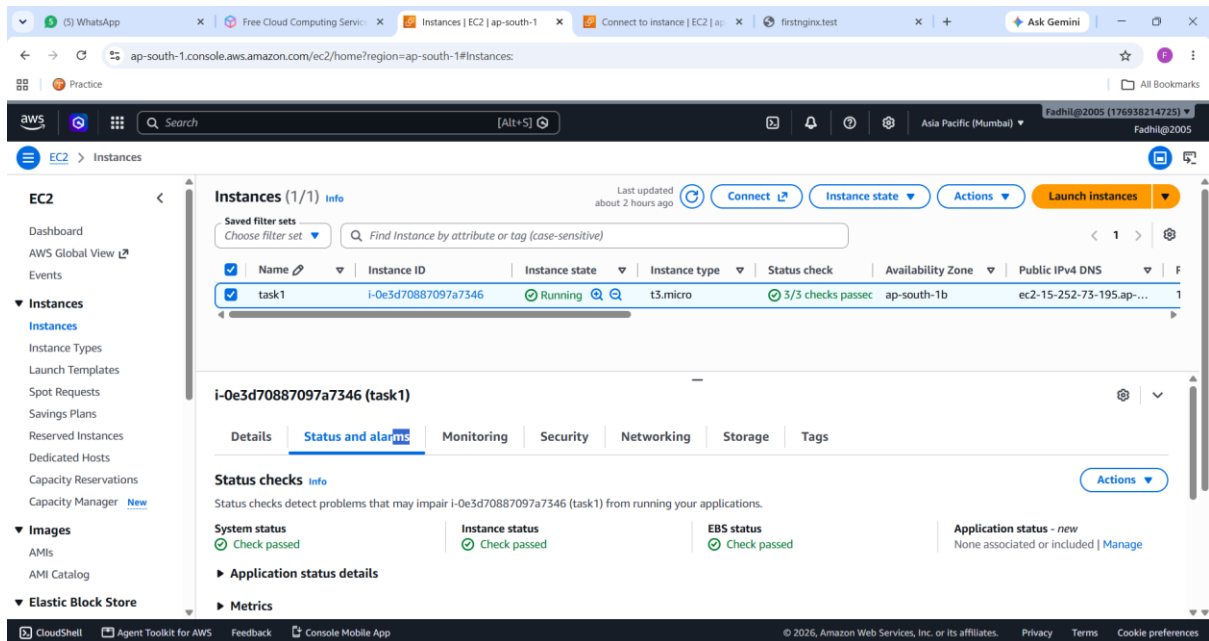


TASK 1: STATIC WEB HOSTING (USING FAKE DOMAIN NAME)

Step1: Launch an instance in EC2 (RHEL)



- And click connect to connect the instance

Step2: Open gitbash and connect the EC2@-user to it

```
- #Cd Downloads
- #ssh -i "awskey.pem" ec2-user@ec2-15-252-73-195.ap-south-1.compute.amazonaws.com
```

Step 3: Install , start and enable the nginx in the machine

```
#sudo yum install nginx -y
#sudo systemctl start nginx
#sudo systemctl enable nginx
```

Step 4: Check status of the nginx

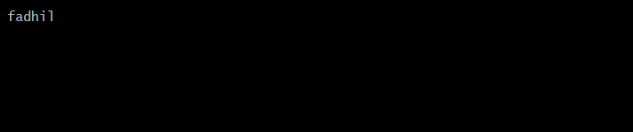
```
#sudo systemctl status nginx
```

Step 5: Edit the index.html file

```
#sudo vi /usr/share/nginx/html/index.html
```

Press i

And edit the index file with our content



The screenshot shows a terminal window with a title bar that reads "ec2-user@ip-172-31-14-110:~". The terminal content shows a shell prompt "hi" followed by the command "my name is fadhil". The prompt "hi" is on the first line, and "my name is fadhil" is on the second line. The terminal has a black background and white text. The window title bar includes standard Linux window controls (minimize, maximize, close) and the user/host information.

Step 6: Reload nginx

```
#sudo systemctl reload nginx
```

- Now open the browser and type the public IP we can see the content we give in the “index.html”
- **Then we want to change the IP address to fake Domain name for the do the step 7**

Step 7: Open the notepad as administrator

- Right click the notepad application and select run as administrator
- Click 'file' > open > search the file 'C:\Windows\System32\drivers\etc\hosts'
- And make sure to change the Text document (*.txt) to All Files
- Add "15.252.73.195 firstnginx.test" to the last line this line set the fake domain name for the public IP address

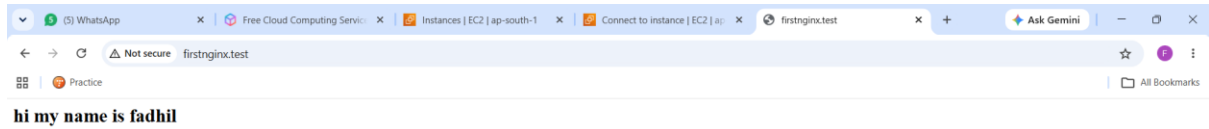
```

# Copyright (c) 1993-2009 Microsoft Corp.
#
# This is a sample HOSTS file used by Microsoft TCP/IP for Windows.
#
# This file contains the mappings of IP addresses to host names. Each
# entry should be kept on an individual line. The IP address should
# be placed in the first column followed by the corresponding host name.
# The IP address and the host name should be separated by at least one
# space.
#
# Additionally, comments (such as these) may be inserted on individual
# lines or following the machine name denoted by a '#' symbol.
#
# For example:
#
#       102.54.94.97       rhino.acme.com          # source server
#       38.25.63.10        x.acme.com             # x client host
#
# localhost name resolution is handled within DNS itself.
#
#       127.0.0.1          localhost
#       ::1                localhost
#       15.252.73.195      firstnginx.test

```

- After setting the IP address to Domain name
- **Verify it by checking the <http://firstnginx.test> in the web browser**

OUTPUT:



TASK 1.1 : The Finance department requires users to authenticate before accessing the website

Commands:

Step 1: Install htpasswd tool

```
#sudo yum install httpd-tools -y
```

- httpd-tools is a package that includes htpasswd, a tool for creating encrypted password files

Step 2: Create the password file with a username and password

bash

```
#sudo htpasswd -c /etc/nginx/.htpasswd fadhil
```

- **htpasswd** — the tool itself
- **-c** — means "create a new file"
- **/etc/nginx/.htpasswd** — the file being created; the leading dot makes it a "hidden" file
- **fadhil** — the username you're creating

Step 3: Open the Nginx config file

```
sudo vi /etc/nginx/nginx.conf
```

- This opens Nginx's main configuration file in the `vi` editor so we can tell Nginx to require login for this site.

Step 4: Find the right section and add authentication

- Inside the file, look for a section that looks like this:

```
server {  
    listen 80;  
    server_name _;  
    root /usr/share/nginx/html;  
  
    location / {  
        index index.html;  
    }  
}
```

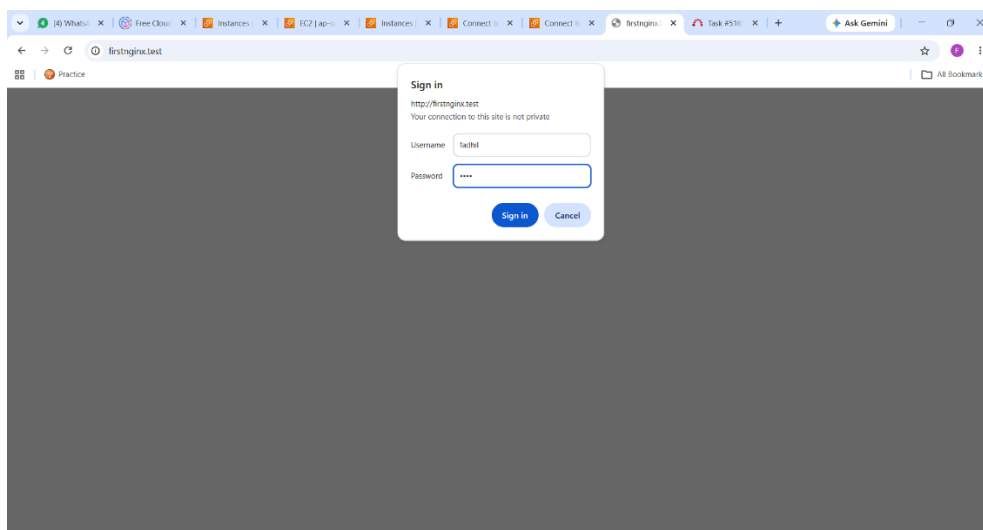
- Press `i` to enter insert mode, then add two new lines inside the location :

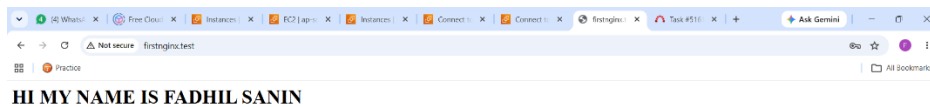
```
location / {  
    index index.html;  
    auth_basic "Restricted Access";  
    auth_basic_user_file /etc/nginx/.htpasswd;  
}
```

Step 5: Reload the nginx

```
#sudo systemctl reload nginx
```

Outputs:





CONCLUSION:

In this task, we installed and configured the Nginx web server on an AWS EC2 instance to host a simple website, using a fake domain name mapped through the hosts file so the site could be accessed by name instead of an IP address. We then added a username and password login using Nginx's Basic Authentication feature, so only authorized users could view the website, successfully completing both the hosting and security requirements of the task